

BASES DE DONNÉES AVANCÉES (NFE204)

Chapitre "Validation d'un document XML (seconde partie) : XML Schema"



DTD

- Sous forme d'une grammaire



simple



rapide à écrire



peu expressive

p.e. *CDATA* : String ? Integer ? Date ?

```
<!ELEMENT NEWSPAPER (ARTICLE+)>
```

```
<!ELEMENT ARTICLE (AUTHOR+,HEADLINE, BYLINE, LEAD, BODY, NOTES*)>
```

```
<!ELEMENT HEADLINE (#PCDATA)>
```

```
<!ELEMENT AUTHOR(#PCDATA)>
```

```
<!ELEMENT BYLINE (#PCDATA)>
```

```
<!ELEMENT LEAD (#PCDATA)>
```

```
<!ELEMENT BODY (#PCDATA)>
```

```
<!ELEMENT NOTES (#PCDATA)>
```

```
<!ATTLIST ARTICLE EDITOR CDATA #IMPLIED>
```

```
<!ATTLIST ARTICLE DATE CDATA #IMPLIED>
```

```
<!ATTLIST ARTICLE EDITION CDATA #IMPLIED>
```

XML Schema

- Formalisme d'expression de schéma plus évolué que les DTD.
→ *moins simple et moins rapide à écrire qu'une DTD.*
- Utilise la syntaxe XML
- Recommandé par le W3C

XML Schema

DTD

```
<!ELEMENT note (to, from, heading, body)>  
<!ELEMENT to (#PCDATA)>  
<!ELEMENT from (#PCDATA)>  
<!ELEMENT heading (#PCDATA)>  
<!ELEMENT body (#PCDATA)>
```

XML Schema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/  
  XMLSchema">  
  <xs:element name="note">  
    <xs:complexType>  
      <xs:sequence>  
        <xs:element name="to" type="xs:string"/>  
        <xs:element name="from" type="xs:string"/>  
        <xs:element name="heading" type="xs:string"/>  
        <xs:element name="body" type="xs:string"/>  
      </xs:sequence>  
    </xs:complexType>  
  </xs:element>  
</xs:schema>
```

XML Schema

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string"/>
        <xs:element name="from" type="xs:string"/>
        <xs:element name="heading" type="xs:string"/>
        <xs:element name="body" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Les éléments : types

<xs:element name="firstname" type="xs:string"/>

<firstname>
Alan
</firstname>

Peut être plus précis en rajoutant des options

```
<element
  abstract = boolean : false
  block = (#all | List of (extension | restriction | substitution))
  default = string
  final = (#all | List of (extension | restriction))
  fixed = string
  form = (qualified | unqualified)
  id = ID
  maxOccurs = (nonNegativeInteger | unbounded) : 1
  minOccurs = nonNegativeInteger : 1
  name = NCName
  nillable = boolean : false
  ref = QName
  substitutionGroup = QName
  type = QName
  {any attributes with non-schema namespace . . .}>
  Content: (annotation?, ((simpleType | complexType)?, (unique | key | keyref)*))
</element>
```

← "Compte..."

← Type

Recomm. W3C

Les éléments : types

Deux sortes de types :

- Types simples : si on veut que l'élément vérifiant le type ne contienne qu'une valeur et pas d'attribut
`<internationalPrice >345,67<internationalPrice >`
- Types complexes : si on veut que l'élément vérifiant le type soit composé de sous-élément(s) ou* contienne des attributs
 - `<internationalPrice currency="dolars">345,67</internationalPrice>`
 - `<internationalPrice currency="EUR" value="345,67"/>`
 - `<internationalPrice>`
`<currency>EUR<currency>`
`<value>345,67<value>`
`</InternationalPrice>`
 - `<internationalPrice date="2009-05-12">`
`<currency>EUR<currency>`
`<value>345,67<value>`
`</InternationalPrice>`

* disjonction non exclusive

Types simples

```
<xs:element name="firstname" type="xs:string"/>
```

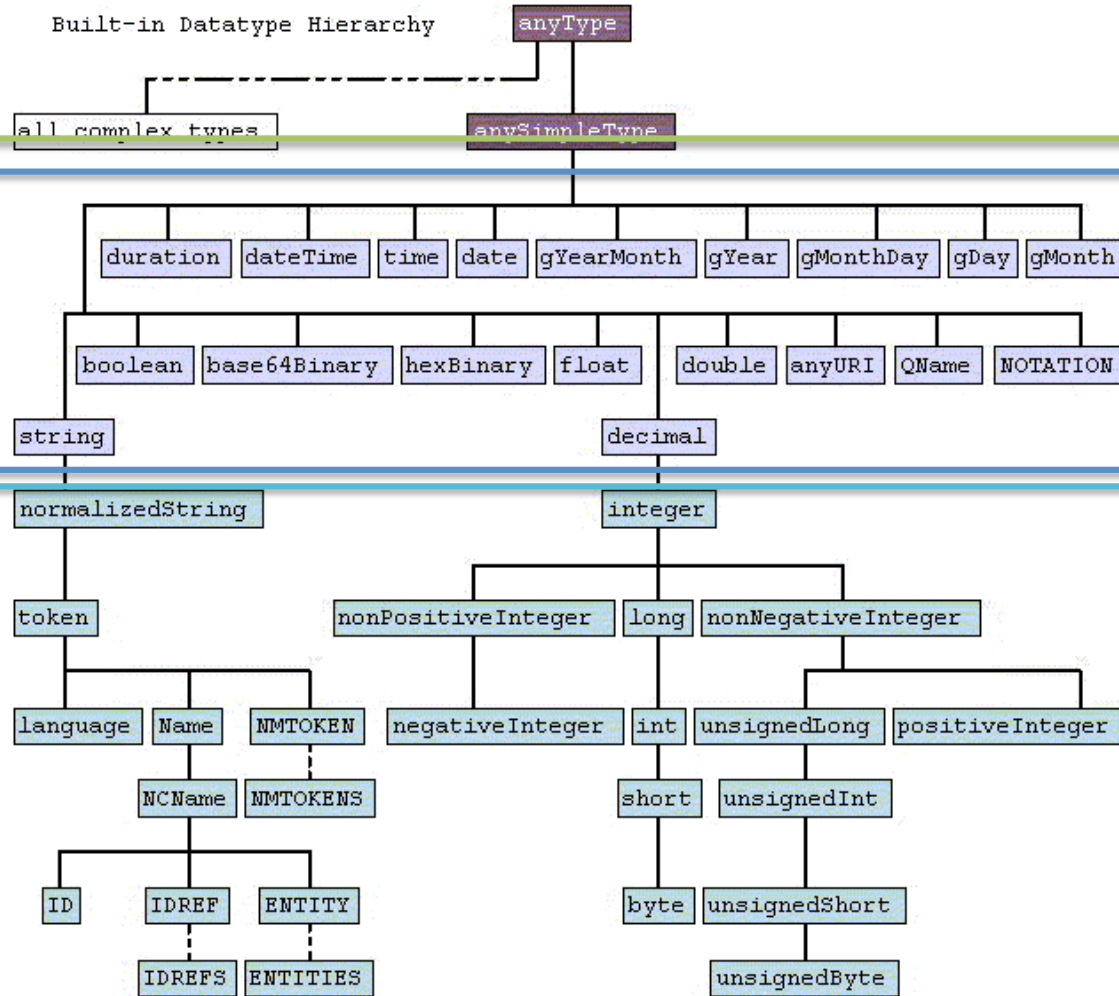
Quelques types possibles :

Type	Description	Commentaire
<i>string</i>	chaîne de caractères	chaîne de caractères
<i>boolean</i>	booléen	'true' 'false' '1' '0'
<i>decimal</i>	décimal	(\+ -)?([0-9](\.[0-9]*)?) \.[0-9]+)
<i>integer</i> (dérivé de <i>decimal</i>)	entier	types dérivés tels que <i>long</i> , <i>non PositiveInteger</i> , <i>nonNegativeInteger</i>
<i>float</i>	flottant	(\+ -)?([0-9](\.[0-9]*)?) \.[0-9]+)([Ee](\+ -)?[0-9]+)? (\+ -)?INF NaN
<i>time</i>		Exemple : 2000-12-12+13:00

Et bien d'autres...

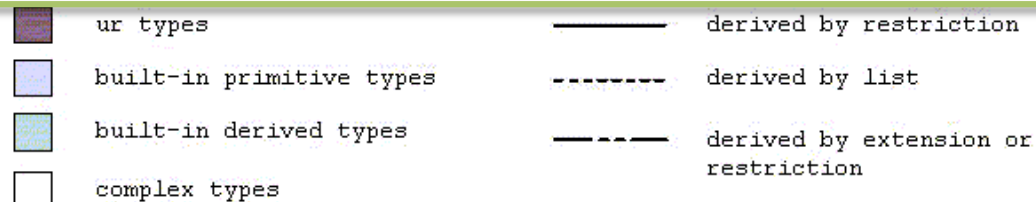
Les types pré-définis

types
pré-définis



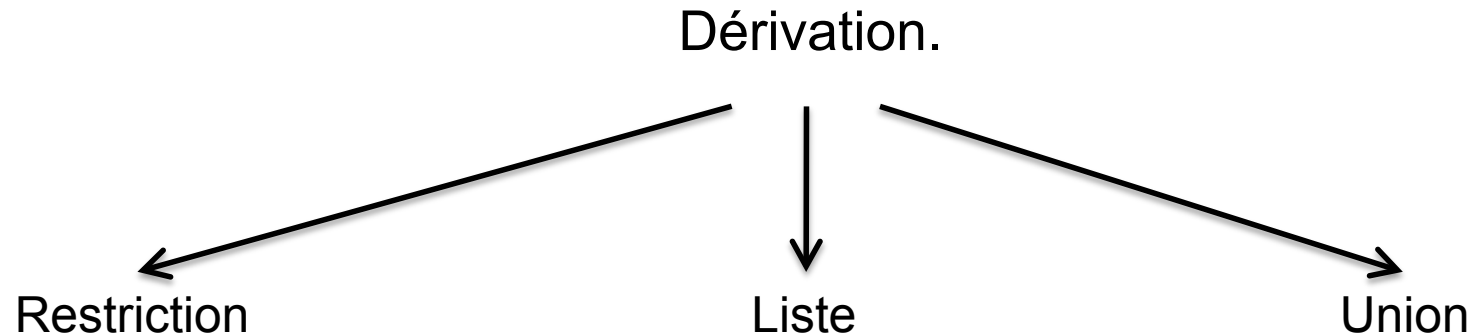
types
primitifs

types dérivés
pré-définis



Types simples : dérivation

On peut aller plus loin qu'un simple type → définition de types dérivés à partir de types pré(cédemment)-définis.



Types simples : dérivation par restriction

A partir de *constraining facets* posant des conditions sur les valeurs.

Dérivation par restriction :

```
<xs:simpleType name='one-hundred-or-less'>
```

```
  <xs:restriction base='xs:integer'>
```

```
    <xs:maxInclusive value='100'>
```

```
  </xs:restriction>
```

```
</xs:simpleType>
```

constraining facet.

Autres possibles pour le type integer :

totalDigits

fractionDigits

pattern

whiteSpace

enumeration

maxExclusive

minInclusive

minExclusive

...

```
<xs:element name="UE_NFA" type="one-hundred-or-less"/>
```

...

Types simples : dérivation par restriction

A partir de *constraining facets* posant des conditions sur les valeurs..

Dérivation par restriction :

```
<xs:simpleType name='CodeIntern'>
```

```
  <xs:restriction base='xs:string'>
```

```
    <xs:pattern value='\c{3}-[A-Z]{2}'/>
```

```
  </xs:restriction>
```

```
</xs:simpleType>
```

constraining facet.



Types simples : dérivation par restriction

Contraindre l'espace des valeurs du type

- length
- minLength
- maxLength
- pattern
- enumeration
- whitespace
- maxInclusive
- maxExclusive
- minExclusive
- minInclusive
- totalDigits

Un seul "jeu" de facettes interprétées différemment selon le type auquel la facette est appliquée.

Par exemple, *length* contraint :

- le nombre de caractères d'un élément *string*
- le nombre d'octets d'un *hexBinary*
- le nombre d'éléments d'une *séquence*

<http://www.w3.org/TR/xmlschema-2/#rf-length>

Types simples : dérivation par restriction

- Constraining facet *length*

```
<xs:simpleType name='productCode'>  
  <xs:restriction base='xs:string'>  
    <xs:length value='8' />  
  </xs:restriction>  
</xs:simpleType>
```

Types simples : dérivation par restriction

- Constraining facet *minLength* et *maxLength*

```
<xs:simpleType name='non-empty-string'>  
  <xs:restriction base='xs:string'>  
    <xs:minLength value='1'/>  
  </xs:restriction>  
</xs:simpleType>
```

```
<xs:simpleType name='form-input'>  
  <xs:restriction base='xs:string'>  
    <xs:maxLength value='50'/>  
  </xs:restriction>  
</xs:simpleType>
```

Types simples : dérivation par restriction

- Constraining facet *pattern*

```
<xs:simpleType name='better-us-zipcode'>  
  <xs:restriction base='xs:string'>  
    <xs:pattern value='[0-9]{5}(-[0-9]{4})?'/>  
  </xs:restriction>  
</xs:simpleType>
```


Types simples : dérivation par restriction

- Constraining facet *enumeration*

```
<xs:simpleType name='holidays'>  
  <xs:restriction base='xs:gMonthDay'>  
    <xs:enumeration value='--01-01'> </xs:enumeration>  
    <xs:enumeration value='--07-04'> </xs:enumeration>  
    <xs:enumeration value='--12-25'> </xs:enumeration>  
  </xs:restriction>  
</xs:simpleType>
```

Types simples : dérivation par restriction

- Constraining facet *whitespace*

```
<xs:simpleType name='token'>
```

```
  <xs:restriction base='xs:normalizedString'>
```

```
    <xs:whiteSpace value='xs:collapse' />
```

```
  </xs:restriction>
```

```
</xs:simpleType>
```

← Indication pour le processeur
qui traitera les éléments
→ pas vraiment du "schéma"...

Types simples : dérivation par restriction

- Constraining facet *maxExclusive*

```
<xs:simpleType name='less-than-one-hundred-and-one'>  
  <xs:restriction base='xs:integer'>  
    <xs:maxExclusive value='101'/> ← "la valeur max est 101 exclus"  
  </xs:restriction>  
</xs:simpleType>
```

Types simples : dérivation par restriction

- Constraining facet *minInclusive*

```
<xs:simpleType name='one-hundred-or-more'>
```

```
  <xs:restriction base='xs:integer'>
```

```
    <xs:minInclusive value='100'>
```

← "la valeur min est 100 inclus"

```
  </xs:restriction>
```

```
</xs:simpleType>
```

Types simples : dérivation par restriction

- Constraining facet *totalDigit*

```
<xs:simpleType name='celsiusBodyTemp'>  
  <xs:restriction base='xs:decimal'>  
    <xs:totalDigits value='3'/>  
    <xs:fractionDigits value='1'/>  
    <xs:minInclusive value='36.4'/>  
    <xs:maxInclusive value='40.5'/>  
  </xs:restriction>  
</xs:simpleType>
```

Types simples : dérivation par liste

```
<xs:simpleType name='listOfFloats'>  
  <xs:list itemType='xs:float'/>  
</xs:simpleType>
```

Exemples de *Constraining facets* applicables :

- length
- maxLength
- minLength

Types simples : dérivation par union

23

```
<xsd:attribute name="size">
  <xsd:simpleType>
    <xsd:union>
```

taille de fonte
exprimée en entiers

```
    <xsd:simpleType>
      <xsd:restriction base="xsd:positiveInteger">
        <xsd:minInclusive value="8"/>
        <xsd:maxInclusive value="72"/>
      </xsd:restriction>
    </xsd:simpleType>
```

```
    <xsd:simpleType>
      <xsd:restriction base="xsd:NMTOKEN">
        <xsd:enumeration value="small"/>
        <xsd:enumeration value="medium"/>
        <xsd:enumeration value="large"/>
      </xsd:restriction>
    </xsd:simpleType>
```

```
  </xsd:union>
</xsd:simpleType>
</xsd:attribute>
```

en small/medium/large

```
<p>
<font size='large'>A header</font>
</p>
<p>
<font size='12'>this is a test</font>
</p>
```

Types complexes : quand ?

Quand passer en type complexe ?

Quand pas un type simple c.-à-d. quand on veut qu'un élément vérifiant le type :

- contienne au moins un attribut ou (non exclusif)
- soit composé de sous-éléments

Exemples (non exhaustif) :

```
<internationalPrice currency="dolars">345,67</internationalPrice>
```

```
<internationalPrice>  
  <currency>EUR<currency>  
  <value>345,67<value>  
</InternationalPrice>
```


2 notations équivalentes

```
<xs:element name="InternationalPrice">  
  <xs:complexType>  
    ...  
  </xs:complexType>  
</xs:element>
```

Plus "intuitif"

```
<xs:element name="InternationalPrice"  
  type="ipriceType" />  
  
<xs:complexType name="ipriceType">  
  ...  
</xs:complexType>
```

Plus clair (plus propre)
si schéma compliqué
p.e. type utilisé à plusieurs
endroits dans le schéma

Types complexes : plusieurs cas

2 principaux cas de figure

- simple content : le contenu de l'élément vérifiant le type est une valeur (pas de sous éléments) + au moins un attribut.

`<internationalPrice currency="dolars">345,67</internationalPrice>`

- complex content : l'élément vérifiant le type est vide ou contient un ou plusieurs sous-élément(s) + éventuellement des attributs

`<internationalPrice>`

`<currency>EUR</currency>`

`<value>345,67</value>`

`</InternationalPrice>`

Types complexes : *simple content*

Simple content : l'élément vérifiant le type contient une valeur + au moins un attribut

```
<internationalPrice currency="dolars">345,67</internationalPrice>
```

```
<xs:element name="internationalPrice">  
  <xs:complexType>  
    <xs:simpleContent>  
      <xs:extension base="xs:decimal">  
        <xs:attribute name="currency" type="xs:string"/>  
      </xs:extension>  
    </xs:simpleContent>  
  </xs:complexType>  
</xs:element>
```

Types complexes : *complex content*

Complexe content : l'élément vérifiant le type est vide ou contient un ou plusieurs sous-élément(s) + éventuellement des attributs

Exemples (non exhaustif) :

`<internationalPrice date="2009-05-12" value="345,67"/>`

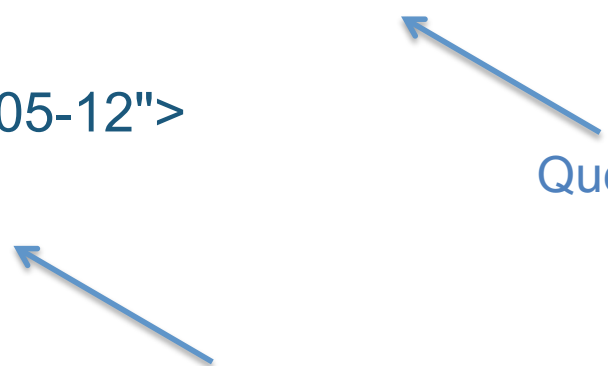
`<internationalPrice date="2009-05-12">`

`<currency>EUR<currency>`

`<value>345,67<value>`

`</InternationalPrice>`

Que des attributs



```
graph TD
    A[Que des attributs] --> B["<internationalPrice date='2009-05-12' value='345,67'/>"]
    C[sous-éléments et attributs] --> D["<internationalPrice date='2009-05-12'>
  <currency>EUR<currency>
  <value>345,67<value>
</InternationalPrice>"]
```

sous-éléments et attributs

Types complexes : *complex content*

Complexe content : l'élément vérifiant le type est vide ou contient un ou plusieurs sous-élément(s) + éventuellement des attributs

`<internationalPrice date="2009-05-12" value="345,67"/>`

Que des attributs :

`<xs:element name="internationalPrice">`

`<xs:complexType>`

`<xs:complexContent>`

`<xs:restriction base="xs:anyType">`

`<xs:attribute name="currency" type="xs:string"/>`

`<xs:attribute name="value" type="xs:decimal"/>`

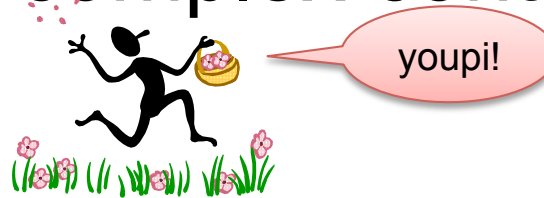
`</xs:restriction>`

`</xs:complexContent>`

`</xs:complexType>`

`</xs:element>`

Types complexes : complex content

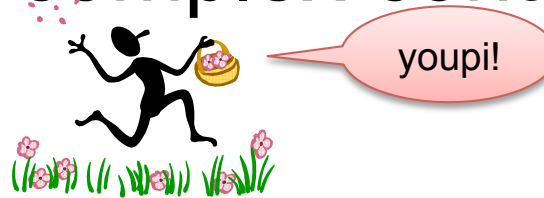


Raccourcis d'écriture

Que des attributs :

```
<xs:element name="internationalPrice">  
  <xs:complexType>  
    <xs:complexContent>  
      <xs:restriction base="xs:anyType">  
        <xs:attribute name="currency" type="xs:string"/>  
        <xs:attribute name="value" type="xs:decimal"/>  
      </xs:restriction>  
    </xs:complexContent>  
  </xs:complexType>  
</xs:element>
```

Types complexes : complex content



Raccourcis d'écriture

Que des attributs :

```
<xs:element name="internationalPrice">
```

```
  <xs:complexType>
```

```
    <xs:attribute name="currency" type="xs:string"/>
```

```
    <xs:attribute name="value" type="xs:decimal"/>
```

```
  </xs:complexType>
```

```
</xs:element>
```

Types complexes : *complex content*

Complexe content : : l'élément vérifiant le type est vide ou contient un ou plusieurs sous-élément(s) + éventuellement des attributs

Exemples :

`<internationalPrice currency="EUR" value="345,67"/>`

Que des attributs



`<internationalPrice date="2009-05-12">`

`<currency>EUR<currency>`

`<value>345,67<value>`

`</InternationalPrice>`

Sous-éléments et attributs



Types complexes : *complex content*

Sous-éléments et attributs ("directement" en raccourcis d'écriture) :

```
<xs:complexType name="InternationalPrice">  
  <xs:sequence>  
    <xs:element name="currency" type="xs:string"/>  
    <xs:element name="value" type="xs:decimal"/>  
  </xs:sequence>  
  <xs:attribute name="date" type="xs:date"/>  
</xs:complexType>
```

```
<internationalPrice date="2009-05-12">  
  <currency>EUR<currency>  
  <value>345,67<value>  
</InternationalPrice>
```

type simples ici mais
aurait pu être des
appels à des types
complexes définis
précédemment

Types complexes : *complex content*

Grammaire plus complète des complex content (*"directement"* en raccourcis d'écriture)

- *Complex content*
 - Sans sous-élément, un *complex content* définit des attributs
 - `<xs:attribute name="n" type="xs:t" use="u" />`
 - n est le nom de l'attribut
 - t est le type XML schema de l'attribut
ENTITY, ID, IDREF(S), string, restriction de string
 - u définit la nécessité de l'attribut
 $u \in \{\text{optional}, \text{required}\}$ et $u = \text{optional}$ par défaut

Types complexes : *complex content*

- *Complex content*
 - Avec sous-éléments et éventuellement des attributs
 - On peut avoir
 - Une séquence (sequence) de sous-éléments dans l'ordre de la définition
 - Exactement un choix (choice) de sous-éléments
 - Un ensemble (all) de sous-éléments dans n'importe quel ordre
 - Suivi d'attributs

```
<xs:all>  
  <xs:element ref="cats"/>  
  <xs:element ref="dogs"/>  
</xs:all>
```

```
<xs:sequence>  
  <xs:choice>  
    <xs:element name="left" type="posType"/>  
    <xs:element name="right" type="posType"/>  
  </xs:choice>  
  <xs:element name="orientation"  
    type="xs:string"/>  
</xs:sequence>  
<xs:attribute name="date" type="xs:date"/>  
<xs:attribute name="iref" type="irefType"/>
```

Types complexes : *complex content*

```
<xs:element name="library" type="libtype" />
<xs:complexType name="libtype">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
    <xs:element name="document" type="doctype" />
    <xs:element name="publisher" type="pubtype" />
  </xs:choice>
</xs:complexType>

<xs:complexType name="doctype">
  <xs:sequence>
    <xs:element name="ISBN" type="xs:string" minOccurs="0" maxOccurs="1"/>
    <xs:element name="author" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
    <xs:element name="title" type="xs:string" minOccurs="1" maxOccurs="1"/>
    <xs:element name="date" type="xs:date" minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="callnumber" type="xs:ID" use="required" />
  <xs:attribute name="language" type="xs:string" use="optional" />
  <xs:attribute name="status" type="statype" default="average" />
  <xs:attribute name="publishedby" type="xs:IDREF" use="optional" />
  <xs:attribute name="related" type="xs:IDREFS" use="optional" />
</xs:complexType>
```

Conclusion

Quel type contenu de type complexe ?

Contenu de l'élément	Attribut	Type	Contenu
Texte	Sans	Simple	<i>pas un type complexe</i>
	Avec	Complexe	simple content
Vide ou sous-éléments	Sans	Complexe	complex content
	Avec		

ID, IDREF

- *Complex content*

- Sans sous élément, un *complex content* définit des attributs

- `<xs:attribute name="n" type="xs:t" use="u" />`

- n est le nom de l'attribut

- t est le type XML schema de l'attribut

- ENTITY, ID, IDREF(S), string, restriction de string

- u définit la nécessité de l'attribut

- $u \in \{\text{optional}, \text{required}\}$ et $u = \text{optional}$ par défaut



Types complexes : *complex content*

```
<xs:element name="library" type="libtype" />
<xs:complexType name="libtype">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
    <xs:element name="document" type="doctype" />
    <xs:element name="publisher" type="pubtype" />
  </xs:choice>
</xs:complexType>

<xs:complexType name="doctype">
  <xs:sequence>
    <xs:element name="ISBN" type="xs:string" minOccurs="0" maxOccurs="1"/>
    <xs:element name="author" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
    <xs:element name="title" type="xs:string" minOccurs="1" maxOccurs="1"/>
    <xs:element name="date" type="xs:date" minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="callnumber" type="xs:ID" use="required" />
  <xs:attribute name="language" type="xs:string" use="optional" />
  <xs:attribute name="status" type="statype" default="average" />
  <xs:attribute name="publishedby" type="xs:IDREF" use="optional" />
  <xs:attribute name="related" type="xs:IDREFS" use="optional" />
</xs:complexType>
```



Prenons un peu de hauteur...

Vérification de documents XML:
intuition des fondements
théoriques

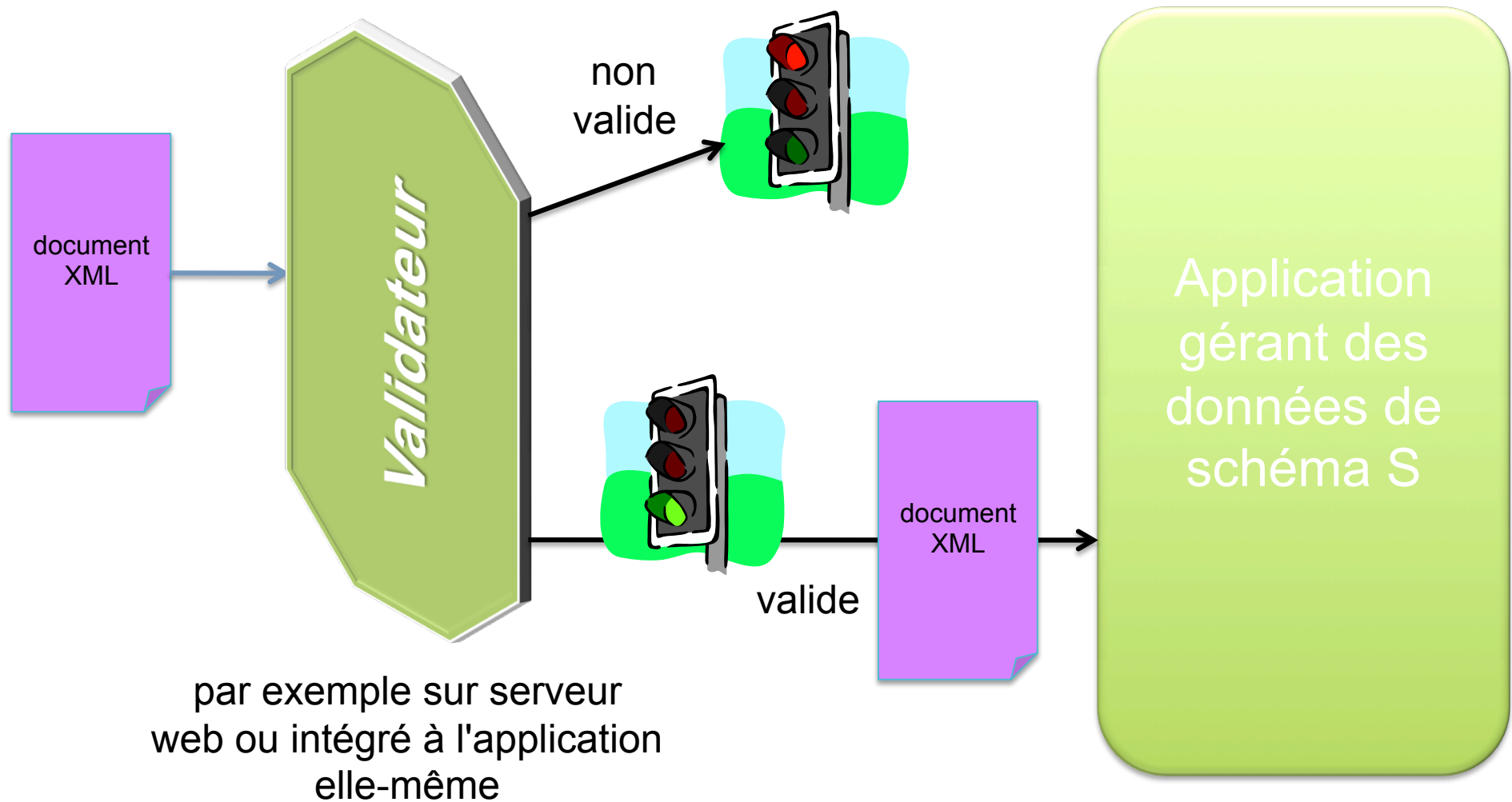
Deux types de vérifications

Deux types de vérification :

- **Vérification dynamique** Un validateur vérifie le type des documents en entrée et éventuellement en sortie de l'application.
Au moment de l'exécution.
- **Vérification statique** On vérifie que le programme (l'application) sachant qu'elle prend un certain type en entrée, renvoie bien un type attendu en sortie.
Avant exécution, à la "compilation" (partiel).

Réf : Section "Typing" de *Web Data Management and Distribution*
Serge Abiteboul, Ioana Manolescu, Philippe Rigaux,
Marie-Christine Rousset, Pierre Senellart

Vérification dynamique



Vérification : intuition

S'appuie sur la théorie des automates.

Un automate décrit un langage.

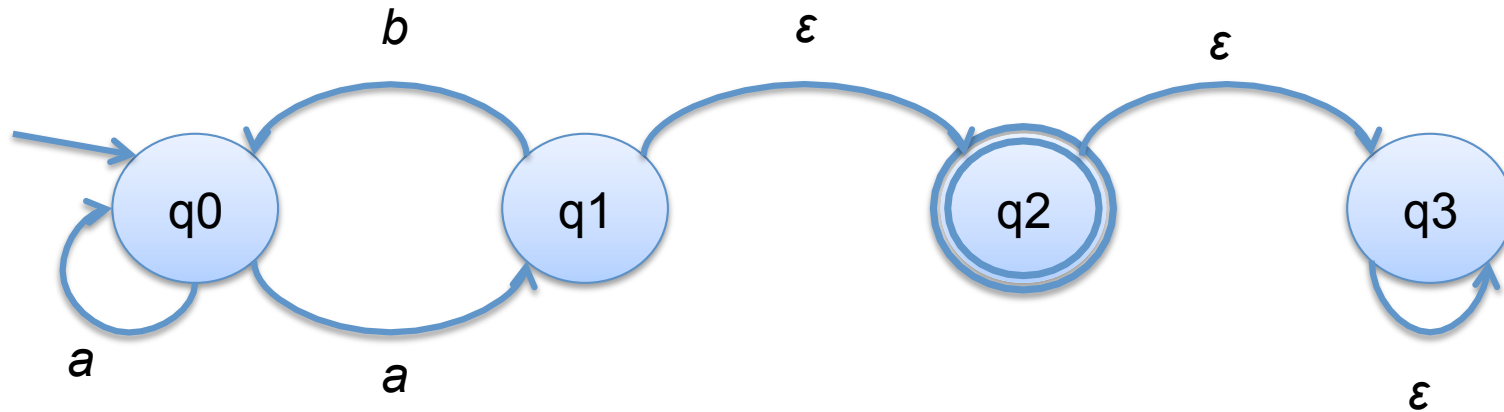
Definition:

A finite state automata is a 5-tuple $(\Sigma, Q, q_0, F, \delta)$ where

- Σ is a finite alphabet;*
- Q is a finite set of states;*
- $q_0 \in Q$ is the initial state;*
- $F \subseteq Q$ is the set of final states; and*
- δ , the transition function, is a mapping from $(\Sigma \cup \{\epsilon\}) \times Q$ to 2^Q (Set of subsets of Q).*

Exercice - Consider the FSA A with $\Sigma = \{a, b\}$, $Q = \{q_0, q_1, q_2, q_3\}$, $F = \{q_2\}$, $\delta(a, q_0) = \{q_0, q_1\}$, $\delta(b, q_1) = \{q_0\}$, $\delta(\epsilon, q_1) = \{q_2\}$, $\delta(\epsilon, q_2) = \{q_3\}$, $\delta(\epsilon, q_3) = \{q_3\}$. Then $abaab$ is in $L(A)$ and aba is not in $L(A)$.

Vérification : intuition



Un cours en ligne concernant les automates :
Jeffrey D. Ullman : <http://infolab.stanford.edu/~ullman/ialc/spr10/spr10.html>

Vérification : intuition

S'appuie sur la théorie des automates d'arbres

1. Document XML = arbre (t),
Schéma traduit en automate d'arbre (A).
2. Validation = vérification que t appartient au langage défini par A

Réfs

Section "Typing" de *Web Data Management and Distribution* de Serge Abiteboul, Ioana Manolescu, Philippe Rigaux, Marie-Christine Rousset, Pierre Senellart
Cours de Mirian Halfeld-Ferrari : <http://www.info.univ-tours.fr/~bouchou/pagesDess/val2004.pdf>

Vérification : intuition

Il existe différentes classes d'automates d'arbres ayant différentes propriétés (par exemple, complexité du processus de vérification).

Selon le langage de schéma qu'on considère (DTD, XMLSchema ou autre), on tombe dans une classe ou une autre.

Plus un langage est expressif, moins il est facile à traiter.

- Eviter les langages de schéma pour lesquels le test de vérification est indécidable ou trop complexe.
- Compromis expressivité – décidabilité/complexité

Réfs

Section "Typing" de *Web Data Management and Distribution* de Serge Abiteboul, Ioana Manolescu, Philippe Rigaux, Marie-Christine Rousset, Pierre Senellart
Cours de Mirian Halfeld-Ferrari : <http://www.info.univ-tours.fr/~bouchou/pagesDess/val2004.pdf>

Vérification statique

- Analyse avant exécution.
- Exemple : XQuery Static Type-Checking in Oracle XML DB

```
SELECT xtab.poref, xtab.usr, xtab.requestor
FROM purchaseorder,
XMLTable('for $i in /PurchaseOrder where $i/costcenter eq "A10" return $i'
PASSING OBJECT_VALUE
COLUMNS poref VARCHAR2(20) PATH 'Reference',
usr VARCHAR2(20) PATH 'User' DEFAULT 'Unknown',
requestor VARCHAR2(20) PATH 'Requestor')
xtab;
```

*Crée une table à partir
d'une requête XQuery effectuée
sur un fichier XML*

*requête XQuery
effectuée sur
PurchaseOrder*

XML Schema associé à PurchaseOrder

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0">
  <xs:element name="PurchaseOrder" type="PurchaseOrderType"/>
  <xs:complexType name="PurchaseOrderType">
    <xs:sequence>
      <xs:element name="Reference" type="ReferenceType"/>
      <xs:element name="Actions" type="ActionsType"/>
      <xs:element name="Reject" type="RejectionType" minOccurs="0"/>
      <xs:element name="Requestor" type="RequestorType"/>
      <xs:element name="User" type="UserType"/>
      <xs:element name="CostCenter" type="CostCenterType"/>
    
```

Vérification statique

- Analyse avant exécution.
- Exemple : XQuery Static Type-Checking in Oracle XML DB

```
SELECT xtab.poref, xtab.usr, xtab.requestor
FROM purchaseorder,
XMLTable('for $i in /PurchaseOrder where $i/costcenter eq "A10" return $i'
PASSING OBJECT_VALUE
COLUMNS poref VARCHAR2(20) PATH 'Reference',
usr VARCHAR2(20) PATH 'User' DEFAULT 'Unknown',
requestor VARCHAR2(20) PATH 'Requestor')
xtab;
```

*Au lieu de CostCenter
(case sensitive)*

XML Schema associé à PurchaseOrder

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" version="1.0">
  <xs:element name="PurchaseOrder" type="PurchaseOrderType"/>
  <xs:complexType name="PurchaseOrderType">
```

ERROR at line 2:

ORA-19276: XPST0005 - XPath step specifies an invalid element/attribute name:
(costcenter)

```
<xs:element name="CostCenter" type="CostCenterType"/>
...
```


Vérification statique

```
SELECT XMLQuery('for $i in ora:view("REGIONS"), $j in ora:view("COUNTRIES")  
  where $i/REGION_ID = $j/REGION_ID and $i/REGION_NAME = "Asia"  
  return $j'  
  RETURNING CONTENT) AS asian_countries  
FROM DUAL;
```

Crée une portion de code XML à partir d'une table relationnelle

```
<ROW> <REGION_ID>1</REGION_ID> <REGION_NAME>blabla </REGION_NAME></ROW>  
<ROW> <REGION_ID>2</REGION_ID> <REGION_NAME>blabla </REGION_NAME></ROW>  
<ROW> <REGION_ID>3</REGION_ID> <REGION_NAME>blabla </REGION_NAME></ROW>  
<ROW> <REGION_ID>4</REGION_ID> <REGION_NAME>blabla </REGION_NAME></ROW>
```

ERROR at line 1:
ORA-19276: XPST0005 - XPath step specifies an invalid element/attribute name:
(REGION_ID)

Produces a static-type-check error, because "ROW" is missing
(\$i/ROW/REGION_ID).